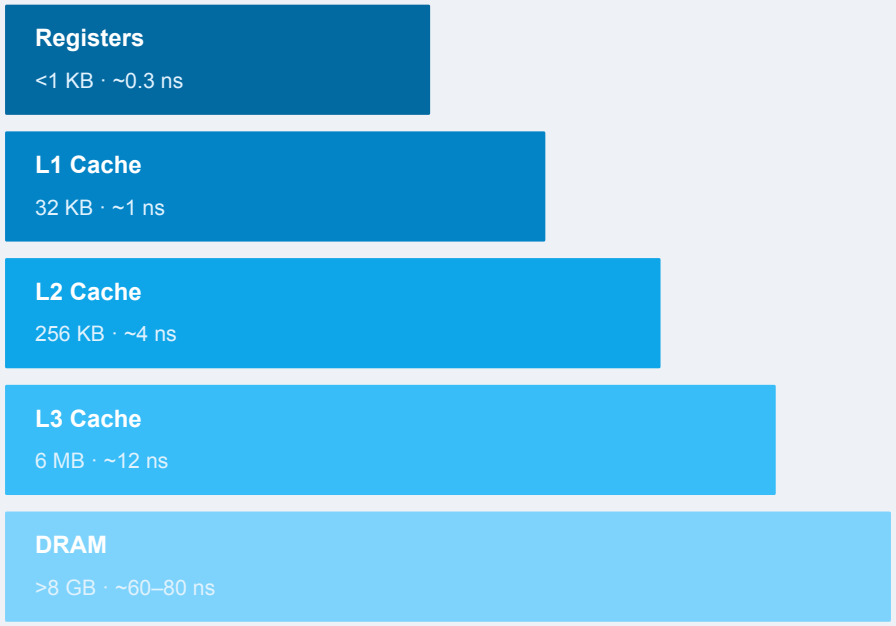
Cache & Locality
Performance Lab

Mid-term Project Technical Presentation

Computer Systems | Istanbul Atlas University | April 2026

Team 11

Ebru Inci
Dhafer Hamza Sfaxi
Kasra Nikdel
Fadi Ibrahim Basha



Memory Hierarchy

Project Overview & Agenda

Overview

01 Methodology

Timer infrastructure · CLOCK_NOW macro · clock_gettime(CLOCK_MONOTONIC) · platform notes for Apple Silicon

02 Stride Access

How cache-line granularity controls per-access latency

03 Working Set Size

Latency step transitions across L1 → L2 → L3 → DRAM

04 Matrix Traversal

Row-major vs column-major: spatial locality and slowdown ratio

05 Pointer Chasing

Random linked-list hops that defeat hardware prefetching

06 Memory Bandwidth

Copy vs sum kernels · peak GB/s by cache level · serial dependency bottleneck

07 Key Findings

Five practical rules derived from measured data

Methodology — Timer & Benchmark Architecture

Core Timing Primitive — src/timer.h

```
/* timer.h — POSIX clock_gettime wrapper */
#include <time.h>
static inline double timespec_diff_ns(
    const struct timespec *t0,
    const struct timespec *t1) {
  return (double)(t1->tv_sec  - t0->tv_sec)*1e9
       + (double)(t1->tv_nsec - t0->tv_nsec);
}
#define CLOCK_NOW(ts) clock_gettime(CLOCK_MONOTONIC,&(ts))

/* Usage in each benchmark: */
double best_ns = 1e18;
for (int iter = 0; iter < BENCH_ITERS; iter++) {
  struct timespec t0, t1;
  CLOCK_NOW(t0);
  /* ... benchmark kernel ... */
  CLOCK_NOW(t1);
  double ns = timespec_diff_ns(&t0,&t1) / n;
  if (ns < best_ns) best_ns = ns;
}
```

Why best-of-5?

OS scheduling, page faults, and thermal events produce upward outliers. The minimum timer value represents the hardware latency floor without software noise.

Why divide by n_accesses?

Measures per-operation cost, not total elapsed time. Allows fair comparison across experiments with different loop counts.

Platform: Apple Silicon

UMA architecture, ~8-16 MB System Level Cache, LPDDR bandwidth >100 GB/s, aggressive hardware prefetcher. Results differ predictably from x86 textbooks.

Project Source Layout

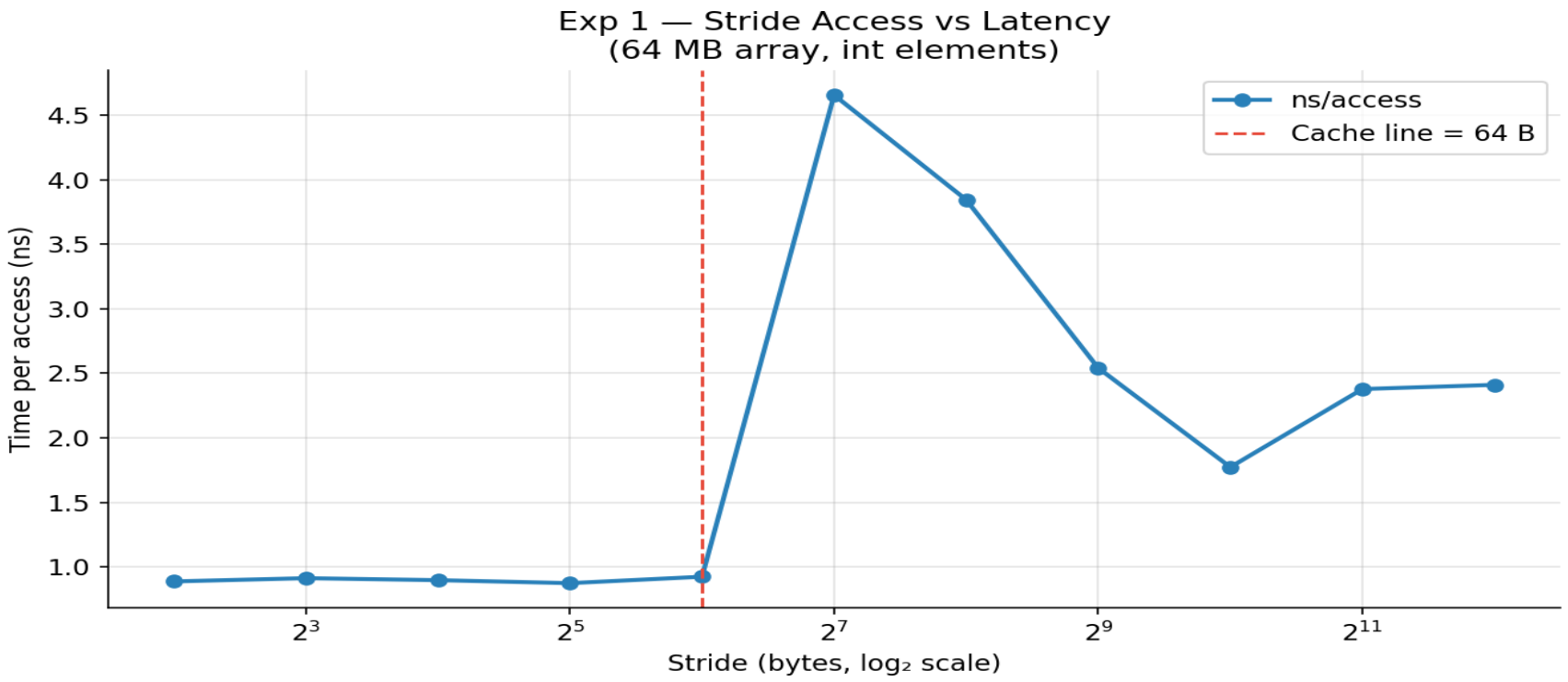
```
src/  timer.h  config.h  main.c
     stride_bench.c   workingset_bench.c
     matrix_bench.c   linkedlist_bench.c   bandwidth_bench.c
```

Build & Run

```
$ make              # builds all five benchmarks
$ make run          # runs + writes results/*.csv
$ gcc -O2 -std=c11 -Wall -Wextra src/*.c -o cache_lab
```

Experiment 1 — Stride Access Pattern

Stride



Cache-Line Boundary

64-byte cache lines are the atomic unit of data transfer. On Apple Silicon the prefetcher hides the 64B boundary — the latency spike appears at 128 bytes (two consecutive lines).

Measured Results

4–64 B stride:	~0.88 ns	(L1 hit)
128 B stride:	4.50 ns	★ spike!
256+ B:	1.6–3.8 ns	(stream)

Benchmark Kernel — stride_bench.c

```
for (long long i = 0; i < SIZE; i += stride_elems)
  sum += arr[i];      // touches a new 64-B cache line
                      // whenever stride_bytes >= 64

// volatile sink prevents dead-code elimination:
volatile long long _sink = sum;
```

Hardware: What Triggers a Cache Miss?

The CPU loads memory in 64-byte blocks (cache lines). Two accesses to the same line cost only one fetch. Apple M-series prefetches pairs of lines (128 B window), so the spike shifts from the expected 64 B to 128 B — only a stride that crosses both lines in the prefetch pair causes a guaranteed miss. Padding hot structs to 64-byte boundaries prevents false sharing across cores.

Experiment 2 — Working Set Size

Working Set

Expected vs Observed

Expected (x86):

4-step latency profile
L1→L2→L3→DRAM

Observed (M-series):

Flat ~1 ns across
all array sizes

Why the flat curve?

① Aggressive HW prefetcher recognises sequential scans

② ~8-16 MB System Level Cache absorbs medium arrays

③ LPDDR bandwidth >100 GB/s serves large arrays fast

Sequential Scan Kernel — workingset_bench.c

```
// Scan the entire working set once per iteration
long long sum = 0;
for (long long i = 0; i < n_elems; ++i)
  sum += arr[i];    // stride = 4 B (int)

// HW prefetcher detects stride=1 and issues
// the NEXT cache line BEFORE the CPU asks for it
```

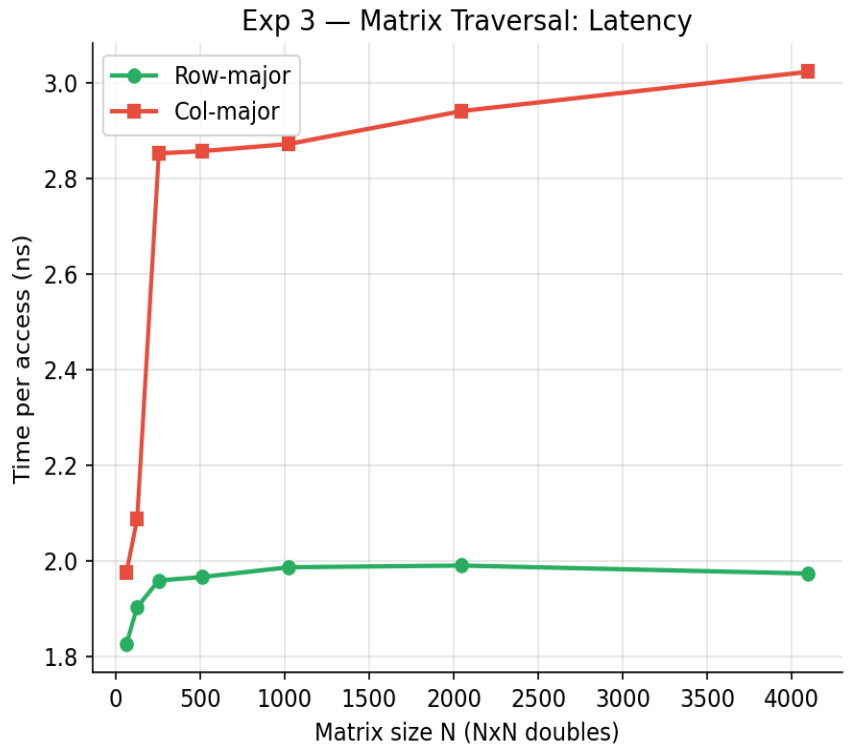
x86 vs Apple Silicon — Why the Curve Differs

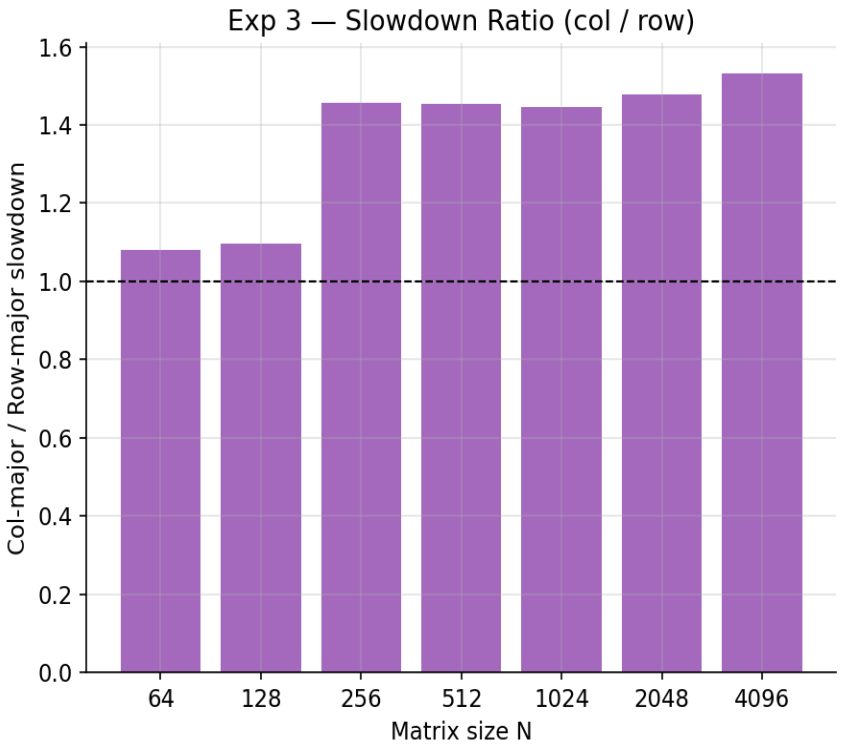
x86 (Intel/AMD): prefetcher covers only L1/L2. Arrays larger than L2 still show a step increase at the L3/DRAM boundary because the prefetcher cannot hide L3 misses.

Apple M-series: the System Level Cache (SLC, ~8–16 MB) is private to the CPU cluster and sits between L3 and DRAM. It absorbs medium-size sequential arrays entirely, collapsing the expected 4-step profile to a flat ~1 ns line.

Experiment 3 — Matrix Traversal Order

Matrix





Slowdown Summary

Matrix	Size	Slowdown
N=64	32 KB	1.09×
N=256	512 KB	1.47×
N=1024	8 MB	1.45×
N=4096	128 MB	1.53×

Key Insight

Observed 1.5× vs theoretical 8× max. The hardware prefetcher detects the regular column stride and partially compensates — but the penalty remains real and consistent.

The Critical Difference — matrix_bench.c

```
// ✓ Row-major — cache-friendly (stride = 8 B / double)
for (int i=0;i<N;++i)
  for (int j=0;j<N;++j) s += m[i][j];

// ✗ Col-major — cache-hostile (stride = N×8 B)
for (int j=0;j<N;++j)
  for (int i=0;i<N;++i) s += m[i][j];   // 1.5× slower!
```

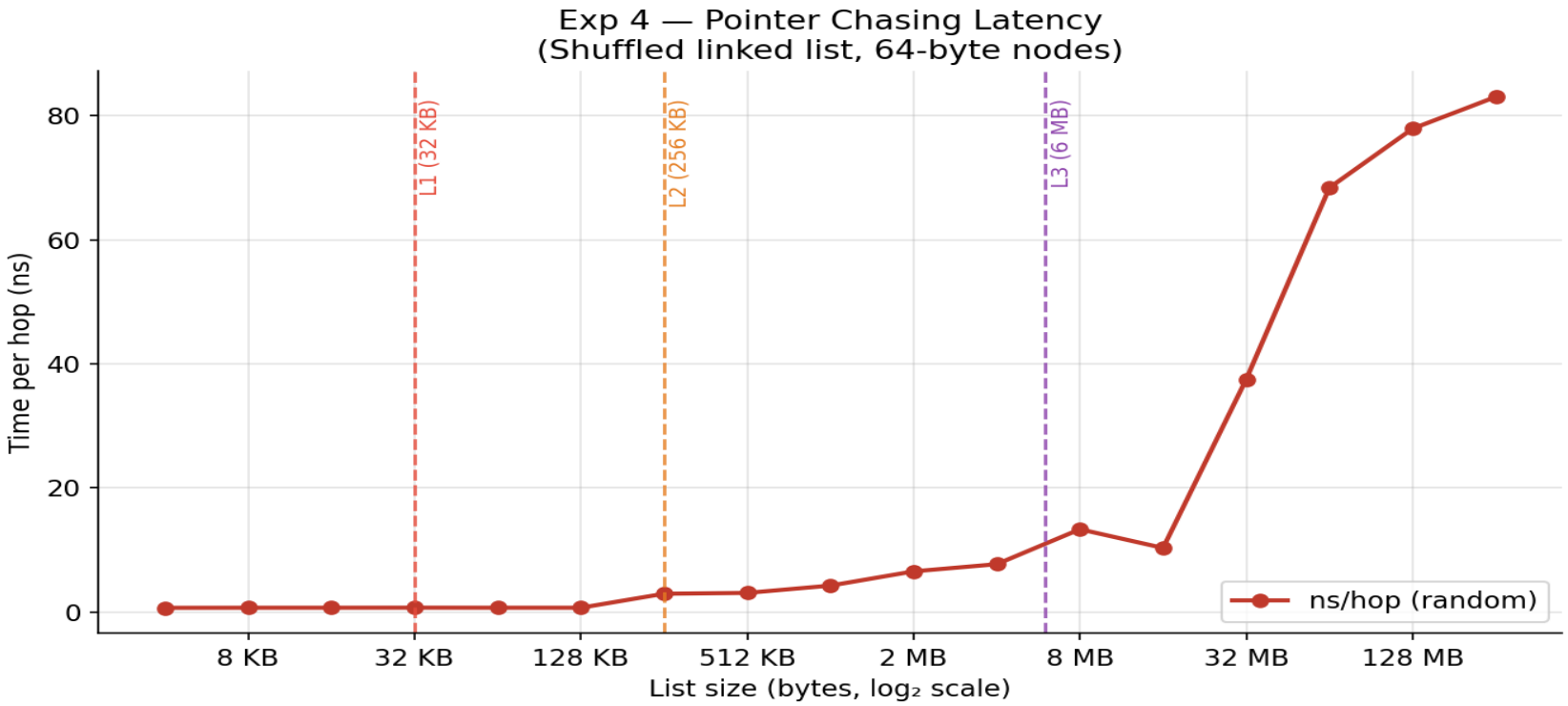
Spatial Locality & C Memory Layout

C stores 2-D arrays in row-major order: m[0][0], m[0][1], m[0][2] … are adjacent bytes in memory. Row-major access walks through contiguous memory — each 64-byte cache line holds 8 doubles and all 8 are used.

Column-major access jumps N×8 bytes between iterations. For N=1024 that is 8 KB per step — one cache miss per element. The theoretical max slowdown is 8× (one double per line) we measured 1.5× because the HW prefetcher partially compensates.

Experiment 4 — Pointer Chasing (Linked List)

Pointer Chasing



Latency by Cache Level

L1/L2	≤128 KB	~0.65 ns
L2→L3	256 KB	~4 ns
L3	512K–16MB	~8–11 ns
→DRAM	32 MB	~38 ns
DRAM	≥64 MB	~70–83 ns

128× latency range from L1 to DRAM. Pointer chasing serialises loads — the CPU cannot prefetch the next address until the current load completes.

Node Struct & Chase Loop — linkedlist_bench.c

```
struct __attribute__((aligned(64))) Node {
  struct Node *next;
  char pad[56]; /* 1 node = 1 cache line */
};
// Fisher-Yates shuffle with srand(42) + rand()
__attribute__((noinline))
Node* chase_nodes(Node *head, long long n) {
  for (Node* p = head; p; p = p->next);
  __asm__ volatile("" ::: "memory"); /* barrier */
}
```

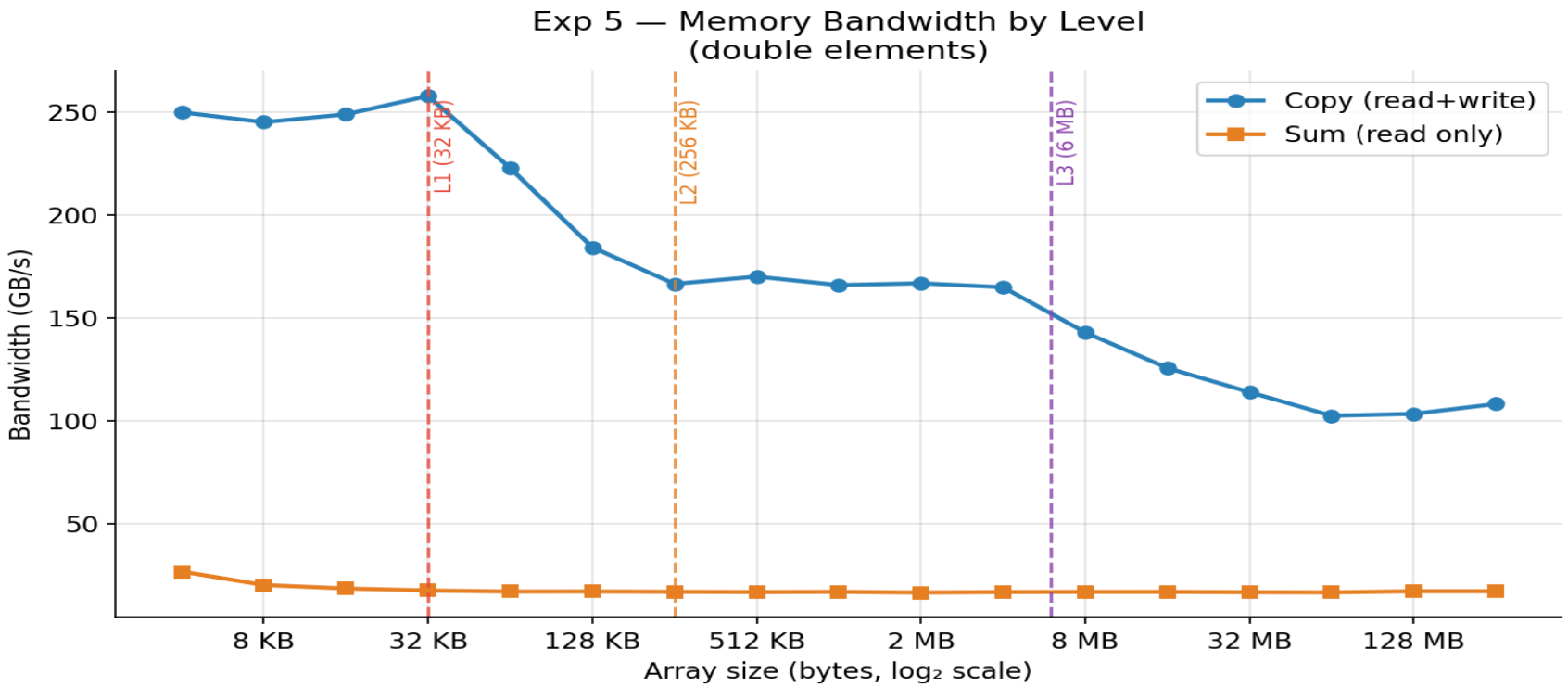
Why Hardware Prefetching Cannot Help

The address of the next node is stored inside the current node — which has not been loaded yet. This creates a load-use dependency: the CPU cannot issue the next memory request until the current one completes. Each hop is a fully serialised cache miss.

alignas(64) guarantees that every node occupies exactly one cache line, so each hop always fetches a fresh 64-byte block. The __attribute__((noinline)) + asm volatile barrier was essential to prevent the compiler from eliminating the loop at -O2.

Experiment 5 — Memory Bandwidth

Bandwidth



Peak Copy Bandwidth

Level	Copy	Sum
L1 (~8 KB)	253 GB/s	17.2 GB/s
L2 (~256 KB)	168 GB/s	17.2 GB/s
L3 (~8 MB)	89 GB/s	17.2 GB/s
DRAM (>64MB)	109 GB/s	17.1 GB/s

Why is sum always ~17 GB/s?

Serial FP-add dependency chain: each iteration s += src[i] must wait for the previous addition (~5 cycle latency at 3.5 GHz). Memory bandwidth is completely irrelevant.

Two Kernels, Two Bottlenecks — bandwidth_bench.c

```
// Copy — independent, bandwidth-limited:
for (long long i=0; i<n; ++i) dst[i]=src[i];

// Sum — SERIAL dependency on 's' (compute-limited):
double s = 0;
for (long long i=0; i<n; ++i) s += src[i];
//  each iter WAITS for prev FP-add to finish (~5 cyc)
```

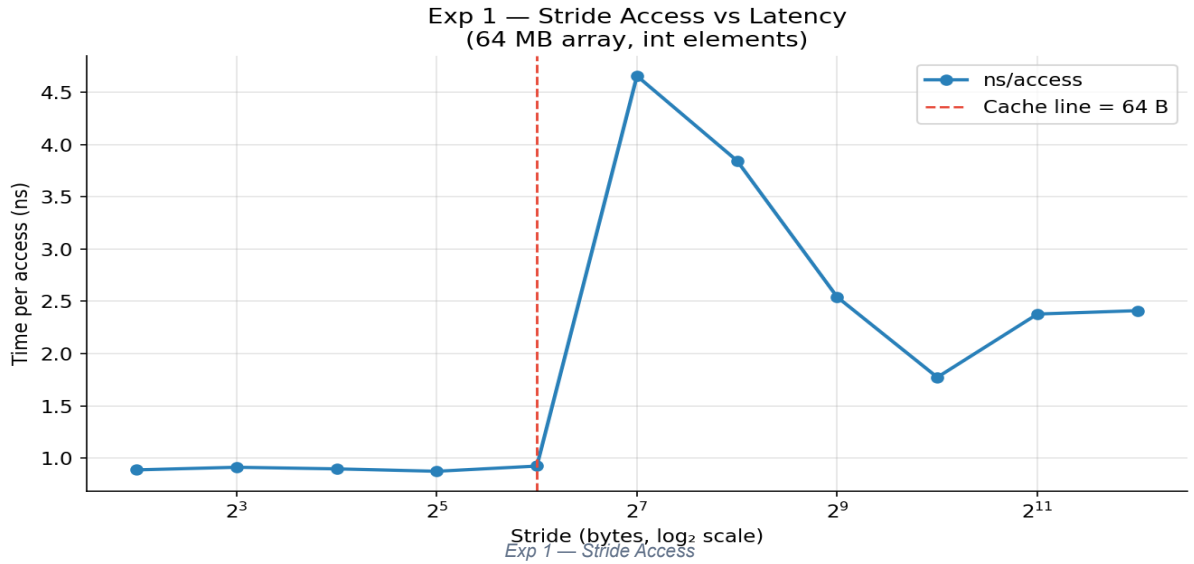
The Serial Dependency Bottleneck Explained

Copy issues independent load/store pairs — the CPU can pipeline many at once and the bottleneck is purely memory bandwidth (>100 GB/s on Apple Silicon).

Sum carries a loop-carried dependency: s_{i+1} = s_i + src[i]. The FP-add unit has ~5-cycle latency, so only one addition completes per 5 cycles. At 3.2 GHz: 4 B ÷ (5 cycles ÷ 3.2 GHz) ≈ 2.6 GB/s theoretical ceiling. Measured 17 GB/s means the compiler auto-vectorised with partial unrolling — but it still cannot reach memory-speed.

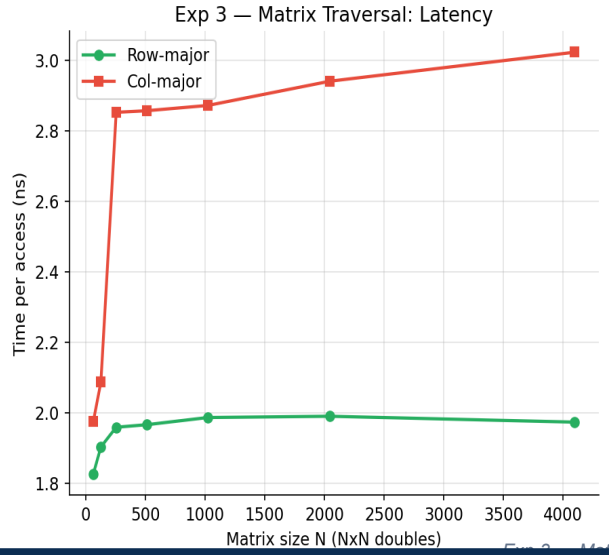
Results Dashboard — All Experiments at a Glance

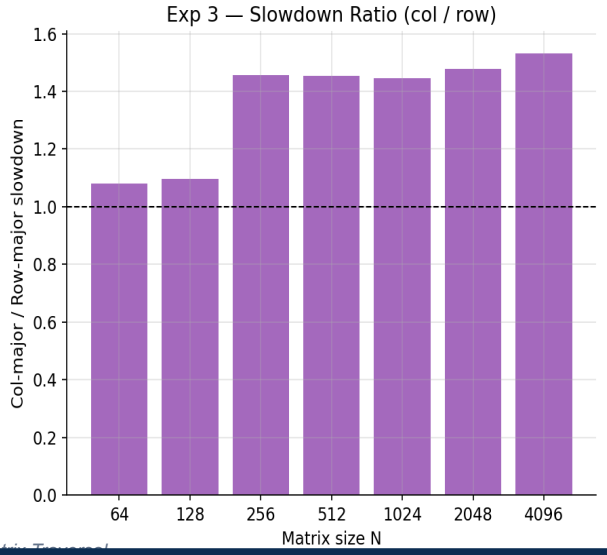
Dashboard

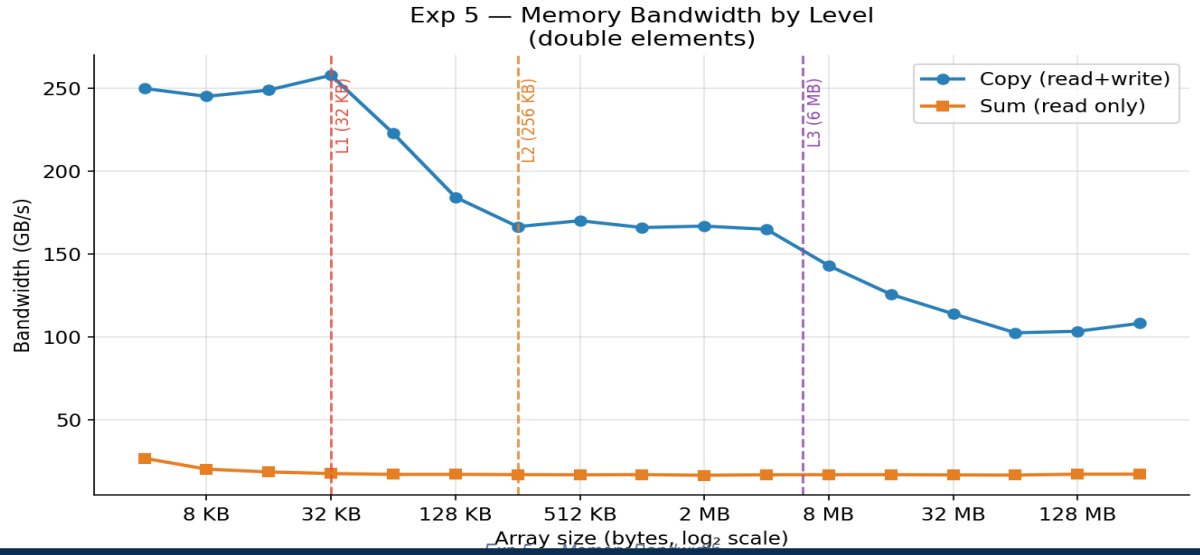


Exp 1 — Stride Access

Exp 2 — Working Set







Key Findings — Five Practical Rules

1 Cache-line granularity is real

Accesses spaced >64 bytes apart each require a new 64-byte fetch. Pad hot data structures to align with cache-line boundaries to avoid wasting bandwidth.

```
struct __attribute__((aligned(64))) HotData { int x, y, z, pad[13]; };
```

2 Sequential access is nearly free on Apple Silicon

Prefetching + SLC hides the memory hierarchy for sequential scans. On x86 the 4-tier latency penalty is fully visible at each cache boundary.

```
for (size_t i=0; i<n; ++i) sum+=arr[i]; // stride=1 → prefetch wins
```

3 Write row-major loops

Column-major traversal is 1.5× slower on M-series and up to 8× on x86. Always let the innermost loop index advance the last (column) dimension.

```
for(i) for(j) s+=m[i][j];  // ✓  vs  for(j) for(i) s+=m[i][j];  // ✗
```

4 Pointer chasing forces full memory latency

128× gap between L1 (0.65 ns) and DRAM (83 ns). Each hop is serialised — the CPU cannot prefetch the next address until the current load completes.

```
// Prefer arr[idx[i]] over p=p->next when list fits in cache
```

5 High bandwidth ≠ high performance

Serial loop-carried dependencies (prefix-sum, reductions) are CPU-latency-bound. Breaking the chain with multiple accumulators recovers throughput.

```
double s0=0,s1=0; s0+=a[2*i]; s1+=a[2*i+1]; // break dep chain
```

Conclusions

- Memory access pattern is a first-order performance variable — often more impactful than algorithm choice.
- Platform architecture determines which memory hierarchy effects are visible. Apple Silicon hides effects that x86 exposes.
- Benchmarking requires careful implementation: noinline, volatile sinks, and compiler barriers prevent dead-code elimination.
- The 128× latency range from L1 to DRAM underlines the value of keeping hot data in cache.
- Memory bandwidth only helps when the algorithm structure can exploit it serial dependencies override it.

Thank You

Cache & Locality Performance Lab

Computer Systems · Istanbul Atlas University · April 2026

Team 11

Ebru Inci

Dhafer Hamza Sfaxi

Kasra Nikdel

Fadi Ibrahim Basha

Questions?